

You are a senior Android architect specializing in Kotlin, Jetpack Compose, Clean Architecture, multi-module Android applications, fintech applications, and production-grade mobile engineering.

I am starting a new Android project called **FinFlow**, a personal finance/expense management application.

I have provided a Supabase PostgreSQL schema for the project. Analyze that schema carefully and use it as the source of truth for the backend/domain structure.

## 1. Existing FinFlow Backend

The database contains these main tables:

- `profiles`
- `accounts`
- `categories`
- `transactions`
- `budgets`
- `goals`

The database also contains these analytics views:

- `monthly_income_expense`
- `monthly_category_spending`
- `budget_usage`

Important relationships:

- `profiles.id` references `auth.users.id`
- `accounts.user_id` references `auth.users.id`
- `categories.user_id` references `auth.users.id`
- `transactions.user_id` references `auth.users.id`
- `transactions.account_id` references `accounts.id`
- `transactions.category_id` references `categories.id`
- `budgets.user_id` references `auth.users.id`
- `budgets.category_id` references `categories.id`
- `goals.user_id` references `auth.users.id`

The database uses Supabase Auth and Row Level Security (RLS).

All user-owned tables have RLS policies restricting access to the authenticated user's `auth.uid()`.

Transactions additionally verify that the referenced account and category belong to the authenticated user.

Budgets additionally verify that the referenced category belongs to the authenticated user.

There are `updated_at` triggers on all main tables.

There is also an `on_auth_user_created` trigger.

When a user signs up, the backend automatically:

1. Creates a profile.
2. Creates a default `Cash` account.
3. Creates default income categories:
  4. Salary
  5. Bonus
6. Creates default expense categories:
  7. Food
  8. Transport
  9. Shopping
  10. Bills
  11. Health
  12. Entertainment

Therefore, the Android client MUST NOT manually create the profile, default Cash account, or default categories after signup.

The analytics views are:

### **monthly\_income\_expense**

Provides:

- month
- total\_income
- total\_expense
- net\_amount

### **monthly\_category\_spending**

Provides:

- month
- category\_id
- category\_name
- category\_color
- total\_amount

### **budget\_usage**

Provides:

- budget\_id

- category\_id
- category\_name
- month
- budget\_amount
- spent\_amount
- remaining\_amount
- usage\_percent

These views should be used by the Android analytics/dashboard layer rather than unnecessarily recreating the aggregation logic on the client.

---

## 2. Architecture Goal

Design a modern, production-grade Android architecture for FinFlow using:

- Kotlin
- Jetpack Compose
- Material 3
- Clean Architecture
- Unidirectional Data Flow / MVI-style state management
- Multi-module Gradle architecture
- Hilt
- Kotlin Coroutines
- Flow / StateFlow
- Supabase
- Room
- DataStore
- Android Keystore / secure storage
- Kotlin Serialization
- Navigation Compose

The application should be designed as a serious portfolio-level fintech application, not as a simple tutorial CRUD app.

Do NOT blindly copy a Flutter folder architecture.

Use proper Android Gradle modules.

---

## 3. Recommended High-Level Modules

Use this structure:

```
FinFlow/
|
|— app/
|
```

```

├── core/
│   ├── common/
│   ├── network/
│   ├── database/
│   ├── datastore/
│   ├── security/
│   ├── designsystem/
│   ├── navigation/
│   └── di/
├── feature/
│   ├── auth/
│   ├── home/
│   ├── accounts/
│   ├── transactions/
│   ├── categories/
│   ├── budgets/
│   ├── goals/
│   └── analytics/
├── build-logic/
└── config/

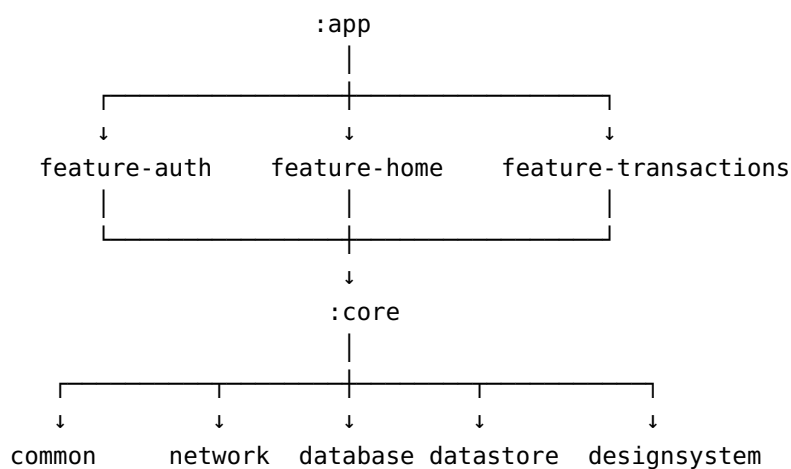
```

Explain why each module exists.

## 4. Dependency Rules

Establish strict dependency rules.

The desired dependency direction is:



Rules:

1. `app` can depend on features and core.
2. Features can depend on core.
3. Core MUST NOT depend on features.
4. Feature modules should NOT directly depend on other feature modules unless there is a very strong architectural reason.
5. Avoid circular dependencies.
6. Shared functionality belongs in core.
7. Business logic belongs in the relevant feature's domain layer.
8. UI components shared across features belong in `core:designsystem`.
9. App-level navigation composition belongs in `app` / core navigation abstractions.
10. Feature-specific navigation remains inside the feature module.

Explain whether any exception is necessary.

---

## 5. Feature Internal Architecture

Every feature should follow:

```
feature-x/  
├── data/  
│   ├── remote/  
│   ├── local/  
│   ├── model/  
│   └── repository/  
├── domain/  
│   ├── model/  
│   ├── repository/  
│   └── usecase/  
├── presentation/  
│   └── feature_name/  
│       ├── Screen.kt  
│       ├── ViewModel.kt  
│       ├── UiState.kt  
│       ├── Event.kt  
│       └── components/  
├── navigation/  
└── di/
```

Use Clean Architecture boundaries:

Presentation



Domain



Data

Explain what each layer is responsible for.

---

## 6. Map Database Tables to Features

Analyze the schema and map each database entity to the correct feature.

Expected starting point:

```
profiles      → auth / profile responsibility
accounts      → accounts feature
categories    → categories feature
transactions  → transactions feature
budgets       → budgets feature
goals         → goals feature

monthly_income_expense
monthly_category_spending
budget_usage
              ↓
              analytics
```

However, do not blindly accept this mapping. Analyze whether `profiles` should be owned by auth or whether a separate profile feature is better.

Explain your final decision.

---

## 7. Auth Architecture

Design the auth flow around Supabase Auth.

Important requirement:

The Android client should NOT insert:

- profile
- default Cash account
- default categories

after signup because the PostgreSQL trigger handles this automatically.

Design:

```
SignupScreen
  ↓
AuthViewModel
  ↓
SignupUseCase
  ↓
AuthRepository
  ↓
Supabase Auth
  ↓
Postgres trigger
  ├── Profile
  ├── Cash Account
  └── Default Categories
```

Also design:

- Login
- Signup
- Logout
- Session restoration
- Forgot password
- Session expiration
- Authentication state
- Token/session handling
- Secure local session storage

Explain what belongs in:

- presentation
- domain
- data
- security/core

---

## 8. Accounts Feature

The database supports:

```
cash
bank
card
wallet
```

Design the Accounts feature around:

- Account list
- Account details
- Create account
- Edit account
- Delete account
- Account balance
- Opening balance

Explain whether balance should be stored/calculated locally, retrieved from backend, or derived from transactions.

Be careful because the schema currently has `opening_balance`, but no explicit current balance column.

Do not invent backend fields.

---

## 9. Transactions Feature

This is one of the most important features.

Design:

```
transactions/  
|  
├─ data/  
├─ domain/  
└─ presentation/  
    ├─ transaction_list/  
    ├─ transaction_detail/  
    └─ add_transaction/
```

The transaction domain should understand:

- account
- category
- income/expense
- amount
- note
- transaction date

Design use cases such as:

```
GetTransactionsUseCase  
GetTransactionUseCase  
CreateTransactionUseCase
```

```
UpdateTransactionUseCase
DeleteTransactionUseCase
```

Also consider:

- filtering
- date ranges
- category filtering
- account filtering
- pagination if appropriate
- sorting
- search
- soft-deleted records

Do not add unsupported backend capabilities without clearly marking them as future enhancements.

---

## 10. Categories Feature

Design categories around:

- Income categories
- Expense categories
- User-created categories
- Color
- Icon
- Soft deletion

The schema enforces unique category names per user/type for non-deleted categories.

Explain how the Android architecture should respect that constraint and how errors should be surfaced to the UI.

---

## 11. Budgets Feature

Budgets are linked to:

```
user
category
month
amount
```

There is also a unique constraint for:

```
user + category + month
```

Design:

```
BudgetList
BudgetDetail
CreateBudget
EditBudget
```

And integrate `budget_usage` into the UI.

The UI should be able to display:

```
Budget
Spent
Remaining
Usage %
```

Do not duplicate the backend calculation unnecessarily.

---

## 12. Goals Feature

Goals contain:

```
name
target_amount
current_amount
target_date
```

Design:

```
GoalList
GoalDetail
CreateGoal
EditGoal
ContributeToGoal
DeleteGoal
```

Think carefully about whether "contribute to goal" should modify `current_amount` directly or whether it should eventually be represented by transactions.

Do NOT silently change the backend model. Explain the tradeoff and recommend an approach compatible with the existing schema.

---

## 13. Analytics Feature

Use the existing Supabase views:

```
monthly_income_expense  
monthly_category_spending  
budget_usage
```

Design:

```
analytics/  
├── data/  
│   ├── remote/  
│   ├── model/  
│   └── repository/  
├── domain/  
│   ├── model/  
│   ├── repository/  
│   └── usecase/  
└── presentation/  
    ├── AnalyticsScreen.kt  
    ├── AnalyticsViewModel.kt  
    ├── AnalyticsUiState.kt  
    └── components/  
        ├── IncomeExpenseChart.kt  
        ├── CategorySpendingChart.kt  
        └── BudgetUsageChart.kt
```

Recommended use cases:

```
GetMonthlySummaryUseCase  
GetCategorySpendingUseCase  
GetBudgetUsageUseCase
```

Explain how Home can consume summary information without creating direct Home → Analytics feature coupling.

---

## 14. Home Architecture

Home should be a composition/orchestration layer.

It should NOT contain database logic.

For example:

```
HomeViewModel
├── GetAccountsUseCase
├── GetRecentTransactionsUseCase
├── GetMonthlySummaryUseCase
└── GetBudgetUsageUseCase
```

Home can show:

- total balance
- monthly income
- monthly expense
- net amount
- recent transactions
- account summary
- budget progress
- goals progress

Explain the cleanest way to compose this data without violating module boundaries.

---

## 15. Room / Offline Strategy

The current backend is Supabase/Postgres.

I want the Android application to be production-grade.

Design an offline-first or cache-first strategy where appropriate.

Discuss:

```
Supabase
  ↑
Repository
  ↑
Room
  ↑
ViewModel
  ↑
Compose
```

For each entity, determine whether Room should be used immediately or whether some data can remain network-only.

Especially analyze:

- transactions
- accounts
- categories
- budgets
- goals

Also explain:

- synchronization
- `updated_at`
- soft deletion
- conflict handling
- incremental sync
- cache invalidation

The backend already uses `updated_at` triggers, so consider how this could support synchronization.

Do not implement a complex sync engine unless justified.

---

## 16. Security

The database uses Supabase RLS.

Explain the correct security architecture.

Important:

```
Android
  ↓
Supabase Auth
  ↓
Session/JWT
  ↓
Supabase
  ↓
Postgres RLS
```

Explain:

- why Android-side `user_id` filtering is not sufficient security
- how RLS protects data
- where session/token management belongs
- what should be stored securely
- what should NEVER be stored in plaintext
- how Android Keystore should be used
- how to handle logout

Do not expose service-role credentials in the Android application.

---

## 17. Design System

Create a reusable Compose design system:

```
core/designsystem/  
  
├─ theme/  
│   ├── Color.kt  
│   ├── Theme.kt  
│   ├── Typography.kt  
│   └── Shapes.kt  
├─ components/  
│   ├── FinFlowButton.kt  
│   ├── FinFlowTextField.kt  
│   ├── FinFlowCard.kt  
│   ├── FinFlowDialog.kt  
│   ├── FinFlowBottomSheet.kt  
│   ├── FinFlowLoader.kt  
│   ├── FinFlowErrorView.kt  
│   └── FinFlowEmptyState.kt  
└─ icons/
```

Explain which components belong here versus feature-specific components.

---

## 18. Navigation

Design type-safe navigation.

I want:

```
App  
├─ Auth Graph  
│   ├── Login  
│   ├── Signup  
│   └── ForgotPassword  
└─ Main Graph  
    ├── Home  
    ├── Accounts  
    └── Transactions
```

```
├─ Budgets
├─ Goals
├─ Analytics
└─ Profile
```

Feature modules should expose their navigation entry points without leaking implementation details.

Explain the recommended Navigation Compose architecture.

---

## 19. Dependency Injection

Use Hilt.

Each feature should have its own DI module where necessary:

```
auth/di/AuthModule.kt
accounts/di/AccountsModule.kt
transactions/di/TransactionsModule.kt
categories/di/CategoriesModule.kt
budgets/di/BudgetsModule.kt
goals/di/GoalsModule.kt
analytics/di/AnalyticsModule.kt
```

Core should contain shared modules:

```
core/di/
├─ SupabaseModule.kt
├─ DatabaseModule.kt
├─ DataStoreModule.kt
└─ CoreModule.kt
```

Explain the dependency graph.

---

## 20. State Management

Use MVI-style/UDF state management.

For each screen use:

```
UiState
Event
ViewModel
```

Example:

```
data class LoginUiState(...)

sealed interface LoginEvent {
    data class EmailChanged(...)
    data class PasswordChanged(...)
    data object LoginClicked : LoginEvent
}
```

And:

```
Compose UI
  ↓ Event
ViewModel
  ↓
UseCase
  ↓
Repository
  ↓
StateFlow
  ↓
Compose UI
```

Explain when to use:

- StateFlow
- SharedFlow
- Channel
- one-shot events

Avoid unnecessary complexity.

---

## 21. Error Handling

Design a consistent error model.

For example:

```
AppError
├─ Network
├─ Authentication
├─ Validation
├─ Database
└─ Unauthorized
```

```
└─ NotFound
└─ Unknown
```

Create a consistent result pattern.

Explain how errors travel:

```
Supabase
↓
Data layer
↓
Domain
↓
ViewModel
↓
UiState
↓
Compose
```

---

## 22. Build Variants

Support:

```
dev
qa
prod
```

Use Android product flavors.

Example:

```
./gradlew assembleDevDebug
./gradlew assembleQaDebug
./gradlew assembleProdRelease
```

Explain how to configure:

- Supabase URL
- Supabase anon/publishable key
- app name
- logging
- debug settings

Never place Supabase service-role keys in the Android app.

---

## 23. Testing Architecture

Design a testing strategy for:

### Unit tests

- UseCases
- ViewModels
- Repositories
- mappers
- validators

### Flow testing

- StateFlow
- events
- async state changes

Use Turbine where appropriate.

### Compose UI tests

Test:

- Login
- Add transaction
- Transaction list
- Budget
- Goal
- Analytics

### Integration tests

Test important flows:

```
Signup
↓
Profile created
↓
Default account/categories available
↓
Create transaction
↓
Analytics updated
```

Explain which tests belong in which module.

---

## 24. AI Module — Future Extension

I also want FinFlow to eventually support an in-app AI financial assistant.

Leave room for:

```
feature/ai/  
├─ data/  
├─ domain/  
├─ presentation/  
└─ di/
```

The future architecture may look like:

```
AI Assistant  
  ↓  
Context Builder  
  ↓  
Transaction Retriever  
  ↓  
Budget Retriever  
  ↓  
Goal Retriever  
  ↓  
Analytics Retriever  
  ↓  
Context / Token Budget  
  ↓  
LLM  
  ↓  
Financial Insight
```

The AI module should NOT directly query arbitrary database tables from the UI.

Design an abstraction that can later retrieve relevant financial context.

Do not over-engineer the AI feature now. Just make the architecture extensible.

---

## 25. Important Architectural Decisions

After analyzing everything, provide a section called:

**Architectural Decisions**

Clearly explain decisions such as:

1. Why multi-module?
  2. Why Clean Architecture?
  3. Why feature-based modules?
  4. Why MVI/UDF?
  5. Why Room?
  6. Why DataStore?
  7. Why Supabase repository abstraction?
  8. Why analytics as a separate feature?
  9. Why Home should orchestrate rather than own business logic?
  10. How RLS fits into security?
  11. How offline sync should work?
  12. How AI can be added later?
- 

## 26. Final Expected Output

Do NOT immediately generate the entire project code.

First give me:

### A. Final project tree

Show the complete recommended directory/module structure.

### B. Module dependency diagram

Show exactly which module depends on which module.

### C. Feature-to-database mapping

Show:

```
Database table/view → Android feature → domain model → repository
```

### D. Data flow

Show:

```
Compose  
→ ViewModel  
→ UseCase  
→ Repository  
→ DataSource  
→ Supabase/Room
```

## E. Authentication flow

Show the complete Supabase Auth + trigger flow.

## F. Offline/sync strategy

Explain how Room + Supabase + `updated_at` should work.

## G. Security architecture

Explain Auth + JWT + RLS + secure storage.

## H. Navigation architecture

Show app graph + feature graphs.

## I. Testing strategy

Show unit/UI/integration test placement.

## J. Future AI architecture

Show how the AI module can be introduced without breaking the architecture.

---

## Critical Constraints

1. Treat the provided Supabase schema as the source of truth.
2. Do not invent database fields.
3. Do not change the backend schema silently.
4. If you recommend a backend schema change, explicitly label it as a recommendation/future improvement.
5. Do not put business logic inside Composables.
6. Do not put database/network calls directly inside ViewModels.
7. Do not let core depend on feature modules.
8. Avoid unnecessary abstractions.
9. Avoid creating a UseCase for every trivial one-line operation unless it improves the domain boundary.
10. Prefer maintainability over architectural ceremony.
11. Keep the architecture suitable for a real fintech-style application.
12. Do not generate the complete implementation yet.
13. First establish and explain the architecture.
14. When there are multiple valid approaches, compare them and select one.
15. Clearly distinguish what is required by the current schema from what is a future enhancement.

The final result should be a **modern, production-grade Jetpack Compose architecture specifically designed for FinFlow and its current Supabase schema**, not a generic Android Clean Architecture template.